4. Приложение Б. Исходный код программы

Полный исходный код проекта содержит более 20 файлов и доступен в полном объеме на электронном носителе, приложенном к курсовой работе. В настоящем приложении представлены ключевые файлы системы.

4.1. Код сервера

main.py:

```
import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from .database import engine, Base
from .routers import auth, podcasts

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Podcast App API", version="1.0.0")

# Автоматически создаем папку для сохранения файлов, если её нет.
os.makedirs("uploads", exist_ok=True)

# Монтируем раздачу статических файлов по адресу /static.
app.mount("/static", StaticFiles(directory="uploads"), name="static")

# Подключаем роутеры.
app.include_router(auth.router)
```

```
app.include_router(podcasts.router)


@app.get("/")
def read_root():
    return {"status": "ok", "message": "Podcast API is working"}


config.py:
import os

# Секретный ключ для подписи токенов JWT. Нужно будет потом убрать.
SECRET_KEY = os.getenv("SECRET_KEY", "your_super_secret_jwt_key_here_change_me_in_production")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24  # Срок действия токена - 24 часа.


database.py:
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "mysql+pymysql://root:*<i51V7CEkgS@localhost:3306/podcast_db"

engine = create_engine(
    DATABASE_URL,
```

```
        pool_pre_ping=True  # Проверка стабильности соединения перед запросом.
    )
    SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
    Base = declarative_base()

    # Генератор для получения сессии БД в end-points.
    def get_db():
        db = SessionLocal()
        try:
            yield db
        finally:
            db.close()
```

dependencies.py:

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.orm import Session
import jwt
from .database import get_db
from .models import User
from .config import SECRET_KEY, ALGORITHM
```

```
    # HTTPBearer автоматически генерирует поле ввода Bearer-токена в Swagger UI
    security_scheme = HTTPBearer()

    def get_current_user(
        credentials: HTTPAuthorizationCredentials = Depends(security_scheme),
        db: Session = Depends(get_db)
    ) -> User:
            """Проверка валидности токена и возвращение текущего пользователя."""
        token = credentials.credentials
        try:
                payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            email: str = payload.get("sub")
            if email is None:
                raise HTTPException(
                    status_code=status.HTTP_401_UNAUTHORIZED,
                    detail="Недействительный токен авторизации",
                    headers={"WWW-Authenticate": "Bearer"},
                )
        except jwt.PyJWTError:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Недействительный токен авторизации",
```

```
            headers={"WWW-Authenticate": "Bearer"},
        )

    user = db.query(User).filter(User.email == email).first()
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Пользователь не найден"
        )
    return user


def get_current_admin(current_user: User = Depends(get_current_user)) -> User:
        """Дополнительная проверка: является ли пользователь администратором."""
    if current_user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Доступ запрещен. Требуются права администратора."
        )
    return current_user
```

models.py:

```
from sqlalchemy import Column, Integer, String, DateTime, func
from .database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(150), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
     role = Column(String(50), default="listener", nullable=False)  # 'listener' или 'admin'.

class Podcast(Base):
    __tablename__ = "podcasts"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(255), nullable=False)
    category = Column(String(100), nullable=False)
    description = Column(String(1000), nullable=True)
    file_path = Column(String(500), nullable=False)  # Путь к MP3 файлу на сервере.
    created_at = Column(DateTime, server_default=func.now())
```

security.py:

```
import bcrypt
import jwt
from datetime import datetime, timedelta, timezone
from .config import SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES


def hash_password(password: str) -> str:
    """Хэширование пароля с солью."""
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt)
    return hashed.decode('utf-8')


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Сравнение чистого пароля с сохраненным хэшем."""
    try:
        return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))
    except Exception:
        return False


def create_access_token(data: dict) -> str:
    """Генерация JWT-токена доступа."""
    to_encode = data.copy()
```

```
        expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
        encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

4.2. Код клиентского мобильного приложения

main.dart:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/auth_provider.dart';
import 'providers/playback_provider.dart';
import 'providers/podcast_provider.dart';
import 'screens/admin_screen.dart';
import 'screens/login_screen.dart';
import 'screens/main_screen.dart';

void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        ChangeNotifierProvider(create: (_) => PodcastProvider()),
        ChangeNotifierProvider(create: (_) => PlaybackProvider()),
```

```
      ],
child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final authProvider = context.watch<AuthProvider>();
    return MaterialApp(
      title: 'Podcasts App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.dark,
        scaffoldBackgroundColor: const Color(0xFF121216),
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF7C4DFF),
          brightness: Brightness.dark,
          primary: const Color(0xFF9E77FA),
          secondary: const Color(0xFF00E676),
          surface: const Color(0xFF1A1A22),
        ),
```

```
appBarTheme: const AppBarTheme(
  backgroundColor: Color(0xFF121216),
  elevation: 0,
  centerTitle: true,
  titleTextStyle: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
    color: Colors.white,
  ),
),
elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: const Color(0xFF7C4DFF),
    foregroundColor: Colors.white,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    textStyle: const TextStyle(
      fontWeight: FontWeight.bold,
      fontSize: 16,
    ),
  ),
),
chipTheme: ChipThemeData(
```

```
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
          ),
          side: BorderSide.none,
        ),
      ),
      home: _getHomeScreen(authProvider),
    );
  }

  Widget _getHomeScreen(AuthProvider authProvider) {
    if (!authProvider.isAuthenticated) {
      return const LoginScreen();
    }
    if (authProvider.role == 'admin') {
      return const AdminScreen();
    }
    return const MainScreen();
  }
}
```